← Retour index Python Niveau III

Python Niveau III — Plan de cours détaillé

Durée : **3 jours (21h)** — progression data complète, de la préparation à la restitution métier.

Jour 1 — Fondations NumPy et pandas

Créneau	Progression	Contenu	Code / livrable	Critère de réussite
09:00–10:30	Base	NumPy : `dtype`, `shape`, indexation, masques booléens.	Notebook de manipulations d'arrays.	Maîtrise des axes et de la vectorisation simple.
10:45–12:30	Base+	Vectorisation, broadcasting, agrégations et contrôle mémoire.	Refactor d'algorithmes à boucles vers NumPy.	Code plus court, plus rapide, résultats identiques.
13:30–15:00	Intermédiaire	pandas : import robuste, parsing de dates, typage, normalisation de colonnes.	Script d'import normalisé.	Données exploitables sans ambiguïtés de types.
15:15–17:00	Consolidation	Nettoyage : NA, doublons, valeurs aberrantes, règles métier.	Pipeline de nettoyage reproductible.	Qualité mesurée avant/après traitement.

Jour 2 — Transformations et KPI

Créneau	Progression	Contenu	Code / livrable	Critère de réussite
09:00–10:30	Intermédiaire	`groupby`, `agg`, `pivot_table`, indicateurs métier.	Tableau KPI journalier et hebdomadaire.	KPI cohérents et vérifiables.
10:45–12:30	Intermédiaire+	Jointures, cardinalités, protection contre les doublons silencieux.	Consolidation fact/référentiel avec `merge(validate=...)`.	Aucune inflation artificielle des volumes.
13:30–15:00	Intermédiaire+	Séries temporelles : index datetime, `resample`, rolling windows.	KPI mensuels + variation MoM/YoY.	Lecture temporelle correcte et exploitable métier.
15:15–17:00	Consolidation	Atelier dirigé : pipeline complet import → nettoyage → KPI.	Notebook structuré et commenté.	Chaîne de calcul reproductible de bout en bout.

Jour 3 — Fiabilité, industrialisation et restitution

Créneau	Progression	Contenu	Code / livrable	Critère de réussite
09:00–10:30	Avancé	Contrôles qualité : taux de NA, contrôles de clés, cohérence des volumes.	Suite de checks automatisables.	Détection d'anomalies avant publication.
10:45–12:30	Avancé	Structurer le code data : fonctions pures, modules, configuration.	Script packagé exécutable en CLI.	Pipeline relançable sur nouvelles données.

13:30–15:00	Projet	Cas fil rouge complet avec contraintes réalistes.	Livrable CSV/Excel + note d'hypothèses.	Résultats argumentés et traçables.
15:15–17:00	Clôture	Revue collective, quiz final, plan de progression individuelle.	Checklist de bonnes pratiques personnalisée.	Autonomie opérationnelle sur projets data intermédiaires.

Bloc théorique transverse (niveau III)

- **Design de pipeline data** : ingestion, standardisation, enrichissement, publication.
- **Qualité statistique** : outliers, biais de sélection, stabilité des agrégats.
- **Fiabilité opérationnelle** : idempotence, reprise sur incident, journalisation des étapes.
- **Performance data** : vectorisation, limitation des copies, chunking sur gros volumes.
- **Traçabilité métier** : hypothèses explicites et définition versionnée des KPI.

Repères de code du niveau III

```
daily = (
    df.set_index("date")
      .resample("D")
      .agg(ca=("prix_net", "sum"))
)

monthly = daily.resample("M").sum()
monthly["delta_m"] = monthly["ca"].pct_change()
```

```
def quality_report(df):
    return {
        "rows": len(df),
        "na_rate": (df.isna().sum() / len(df)).to_dict(),
        "duplicates": int(df.duplicated().sum())
    }
```

Stack : Python 3.11+, NumPy, pandas, VS Code/Jupyter.

Détails techniques approfondis (niveau III)

Qualité de données

- Définir des seuils d'acceptation: taux de NA max, taux de doublons max, bornes métier.
- Standardiser les conventions de nommage de colonnes avant tout enrichissement.
- Contrôler la stabilité des agrégats d'un run à l'autre (variance anormale).
- Séparer anomalies bloquantes et anomalies non bloquantes avec journal de décision.

Performance des pipelines

- Mesurer le coût des étapes critiques (read, merge, groupby, export) sur volume réaliste.
- Réduire les copies mémoire: éviter chaînes de transformations inutiles et colonnes temporaires superflues.
- Adapter les `dtype` pour limiter l'empreinte sans dégrader la précision métier.
- Découper les traitements volumineux en lots idempotents si nécessaire.

Gouvernance des KPI

- Documenter source primaire, règles de calcul, exclusions et responsabilité métier.
- Versionner le calcul d'indicateur pour expliciter les ruptures de série historique.
- Mettre en place une validation croisée (comptabilité, finance, opérations).
- Publier avec une fenêtre de recalcul contrôlée pour les données tardives.

Glossaire opérationnel

- **Schema-on-read** : interprétation du schéma à la lecture des données.
- **Cardinalité contrôlée** : relation de jointure explicitement validée.
- **Rolling window** : calcul glissant sur une fenêtre temporelle mobile.
- **Baseline KPI** : valeur de référence utilisée pour détecter les écarts.
- **Audit trail** : historique horodaté des traitements et décisions.

Définitions techniques complémentaires (niveau III)

- **Data profiling** : analyse initiale des distributions, manquants, cardinalités et anomalies.
- **Schema drift** : évolution inattendue du schéma source (colonnes, types, formats).
- **Skew** : déséquilibre de distribution pouvant dégrader les agrégations ou performances.
- **Partitionnement logique** : découpage des traitements par période, région ou domaine métier.

- **Window function** : calcul contextuel sur fenêtre ordonnée sans perdre le niveau ligne.
- **Reconciliation** : rapprochement entre résultat calculé et source de référence métier.
- **Data quality gate** : seuil bloquant empêchant la publication d'un livrable non conforme.

Approfondissement technique (niveau III)

Modélisation et fiabilité des transformations

- Stabiliser les clés métier en amont pour sécuriser jointures et agrégations.
- Versionner les règles de transformation pour expliquer chaque rupture de série.
- Séparer clairement logique de calcul, contrôles qualité et publication des outputs.

Performance et passage à l'échelle

- Mesurer CPU, mémoire et temps sur des échantillons représentatifs avant optimisation.
- Réduire les conversions de type répétées en consolidant le typage dès l'ingestion.
- Découper les pipelines en étapes idempotentes pour reprise partielle après incident.

Qualité de restitution métier

- Associer chaque KPI à une fiche de définition, propriétaire et règle de recalcul.
- Afficher systématiquement période couverte, date de calcul et taux de données rejetées.
- Réaliser une validation croisée avec au moins une source externe de contrôle.

```
def enforce_quality_gate(report: dict) -> None:
    if report["na_rate_global"] > 0.03:
        raise ValueError("quality gate KO: trop de valeurs manquantes")
    if report["duplicate_rate"] > 0.01:
        raise ValueError("quality gate KO: trop de doublons")
```